

DIA 1

hermes chat -t terminal,file -q "tu pregunta de código aquí"

```
• You: Crea un if/else de ruby on rails 3
♦ Hermes: Para crear un 'if/else' en Ruby on Rails 3, puedes usarlo tanto en tus **Modelos/Controladores** (Ruby puro) como en tus **Vistas** (ERB).

Aquí tienes los ejemplos:

### 1. En un Controlador o Modelo (Ruby puro)
Es la sintaxis estándar de Ruby. No requiere 'then', pero sí un 'end' para cerrar el bloque.

```ruby
if @usuario.activo?
 flash[:notice] = "Usuario activo"
else
 flash[:alert] = "Usuario inactivo"
end
```

### 2. En una Vista (ERB)
En las vistas de Rails 3, se utiliza '<%= %>' para lógica y '<%= %>' si necesitas imprimir algo en el HTML.

```erb
<%= if @usuario.activo? %>
 <p>El usuario está activo.</p>
<%= else %>
 <p>El usuario no está activo.</p>
<%= end %>
```

### Un consejo útil: 'elsif'
Si necesitas evaluar más de una condición, utiliza 'elsif':

```ruby
<%= if @usuario.admin? %>
 <%= link_to "Panel de Admin", admin_path %>
<%= elsif @usuario.moderador? %>
 <%= link_to "Panel de Moderador", mod_path %>
<%= else %>
 <p>Acceso restringido.</p>
<%= end %>
```

**Nota importante:** Rails 3 ya tiene sus años, pero la sintaxis básica de 'if/else' en Ruby no ha cambiado, por lo que esto funcionará perfectamente en cualquier versión moderna de Rails también.
```

hermes chat -q "revisa este código y dime si está bien"

```
hermes chat -q "revisa este código y dime si está bien:
import java.util.ArrayList;
import java.util.List;

// Clase principal que representa a un graduado
class Graduado {
    private String idGraduado;
    private String nombre;
    private String carrera;
    private int anoGraduacion;
    private boolean empleado;
    private String empresaActual;

    // Constructor
    public Graduado(String id, String nombre, String carrera, int ano, boolean empleado, String empresa) {
        this.idGraduado = id;
        this.nombre = nombre;
        this.carrera = carrera;
        this.anoGraduacion = ano;
        this.empleado = empleado;
        this.empresaActual = empresa;
    }

    // Métodos de acceso
    public String getIdGraduado() { return idGraduado; }
    public String getNombre() { return nombre; }
    public String getCarrera() { return carrera; }
    public int getAnoGraduacion() { return anoGraduacion; }
    public boolean getEmpleado() { return empleado; }
    public String getEmpresaActual() { return empresaActual; }

    // Métodos de negocio
    public void setEmpleado(boolean empleado) { this.empleado = empleado; }
    public void setEmpresaActual(String empresa) { this.empresaActual = empresa; }

    // Métodos de utilidad
    public boolean esEmpleado() { return empleado; }
    public boolean esEstudiante() { return !empleado; }
    public boolean esDesempleado() { return !empleado; }

    // Métodos de cálculo
    public double calcularTasaEmpleabilidad() {
        if (listaGraduados.isEmpty()) return 0.0;
        long empleados = listaGraduados.stream()
            .filter(Graduado::isEmpleado)
            .count();
        return ((double) empleados / listaGraduados.size()) * 100;
    }
}
```

hermes

Como indicué en el análisis anterior, el código es funcionalmente correcto y compila bien, pero desde una perspectiva de Ingeniería de Software Senior, tiene varios puntos de mejora para que sea un código profesional, mantenible y robusto.

Aquí tienes un resumen de por qué está 'bien' para un ejercicio académico, pero qué necesita cambiar para un entorno real (especialmente en un sistema universitario):

Puntos Fuertes (Lo que está bien)

1. Lógica Clara: El código es legible y sigue la lógica de negocio básica (agregar graduados, buscar por carrera y calcular estadísticas).
2. Uso correcto de Colecciones: Utilizar List<Graduado> es la práctica estándar.
3. Encapsulamiento básico: Los atributos están definidos como private, lo cual es correcto.

Lo que debes mejorar (Bugs y malas prácticas)

1. Riesgo de ArithmeticException (División por cero):
 - En `calcularTasaEmpleabilidad()`, si la lista está vacía (`listaGraduados.size() == 0`), la división `empleos / listaGraduados.size()` fallará.
 - Solución: Agrega un chequeo `if (listaGraduados.isEmpty())` antes de calcular la tasa.
2. Acoplamiento de presentación (Separación de responsabilidades):
 - La clase `Graduado` no debería tener `System.out.println` dentro. En una arquitectura profesional, el objeto debe contener los datos, pero no decidir cómo imprimirse en consola.
 - Solución: Implementa el método `toString()` o crea una clase `GraduadoPrinter`.
3. Uso de tipos primitivos/datos inseguros:
 - El campo `empresaActual` es `null` si el graduado no tiene empleo. Esto es peligroso para los reportes.
 - Solución: Usa un `Optional<String>` o permite que sea `null` y maneja esa lógica en el Getter.
4. Estilo Imperativo vs. Declarativo:
 - Estás usando ciclos `for` tradicionales. En Java moderno (`8+`), es preferible usar `Streams` para filtrar y contar, ya que es más seguro y reduce errores de lógica.

Ejemplo de cómo sería un código más profesional (Refactorizado):

```
java
public double calcularTasaEmpleabilidad() {
    if (listaGraduados.isEmpty()) return 0.0;

    long empleados = listaGraduados.stream()
        .filter(Graduado::isEmpleado)
        .count();

    return ((double) empleados / listaGraduados.size()) * 100;
}
```

Veredicto: El código es correcto para aprender Java y hacer pruebas rápidas. Si es para un proyecto real de la UJMD, te recomiendo aplicar los cambios de validación de lista vacía y separación de la lógica de impresión para evitar errores en producción.

¿Te gustaría que te escriba una versión refactorizada aplicando estos cambios senior?

hermes chat -t file -q "Eres un senior developer con experiencia en Java y sistemas universitarios. Revisa el siguiente fragmento de código del módulo de Graduados de la UJMD. Identifica: bugs potenciales, mejoras de legibilidad, y si hay patrones que deberían refactorizarse. Explica cada hallazgo con el porqué. import java.util.ArrayList;

import java.util.List;

// Clase principal que representa a un graduado

class Graduado {

private String idGraduado;

private String nombre;

private String carrera;

private int anoGraduacion;

private boolean empleado;

private String empresaActual;

// Constructor

```
public Graduado(String idGraduado, String nombre, String carrera, int
anoGraduacion, boolean empleado, String empresaActual) {

    this.idGraduado = idGraduado;

    this.nombre = nombre;

    this.carrera = carrera;

    this.anoGraduacion = anoGraduacion;

    this.empleado = empleado;

    this.empresaActual = empresaActual;

}
```

// Métodos Getters y Setters

```
public String getNombre() { return nombre; }

public String getCarrera() { return carrera; }

public boolean isEmpleado() { return empleado; }

public void setEmpleado(boolean empleado) { this.empleado = empleado; }

public void setEmpresaActual(String empresaActual) { this.empresaActual =
empresaActual; }
```

// Método para mostrar la información del graduado

```
public void mostrarInformacion() {

    System.out.println("ID: " + idGraduado);

    System.out.println("Nombre: " + nombre);

    System.out.println("Carrera: " + carrera);

    System.out.println("Año de Graduación: " + anoGraduacion);

    System.out.println("Estado Laboral: " + (empleado ? "Empleado en " +
empresaActual : "En búsqueda activa"));

}
```

```

        System.out.println("-----");
    }
}

// Clase controladora para administrar el módulo de graduados
public class ModuloGraduados {

    private List<Graduado> listaGraduados;

    public ModuloGraduados() {
        listaGraduados = new ArrayList<>();
    }

    public void agregarGraduado(Graduado graduado) {
        listaGraduados.add(graduado);

        System.out.println("Graduado " + graduado.getNombre() + " agregado al
sistema.");
    }

    public void buscarPorCarrera(String carrera) {
        System.out.println("\n--- Graduados de la carrera: " + carrera + " ---");

        boolean encontrado = false;

        for (Graduado g : listaGraduados) {
            if (g.getCarrera().equalsIgnoreCase(carrera)) {
                g.mostrarInformacion();

                encontrado = true;
            }
        }
    }
}

```

```
}  
  
if (!encontrado) {  
    System.out.println("No se encontraron graduados en esta carrera.");  
}  
}
```

```
public void mostrarEstadisticasEmpleo() {  
    int empleos = 0;  
    for (Graduado g : listaGraduados) {  
        if (g.isEmpleado()) {  
            empleos++;  
        }  
    }  
  
    double tasaEmpleo = ((double) empleos / listaGraduados.size()) * 100;  
    System.out.println("\n--- Estadísticas de Empleabilidad ---");  
    System.out.println("Total de Graduados: " + listaGraduados.size());  
    System.out.println("Tasa de Empleabilidad: " + tasaEmpleo + "%");  
}
```

```
// Método principal (Main)
```

```
public static void main(String[] args) {
```

```
    ModuloGraduados modulo = new ModuloGraduados();
```

```
    // Registrar nuevos graduados
```

```
    modulo.agregarGraduado(new Graduado("G001", "Ana Pérez", "Ingeniería en  
Sistemas", 2025, true, "Tech Solutions"));
```

```
modulo.agregarGraduado(new Graduado("G002", "Carlos Gómez", "Licenciatura en Administración", 2024, false, "Ninguna"));
```

```
modulo.agregarGraduado(new Graduado("G003", "Lucía Martínez", "Ingeniería en Sistemas", 2026, true, "Software Corp"));
```

```
// Consultas
```

```
modulo.mostrarEstadisticasEmpleo();
```

```
modulo.buscarPorCarrera("Ingeniería en Sistemas");
```

```
}
```

```
}"
```

```
hermes chat -t file -q "Eres un senior developer con experiencia en Java y sistemas universitarios. Revisa el siguiente fragmento de código del módulo de Graduados de la UMD. Identifica: bugs potenciales, mejoras de legibilidad, y si hay patrones que deberían refactorizarse. Explica cada hallazgo con el porqué. Importa java.util.ArrayList; import java.util.List; }

// preparing delegate task.
// delegata: Revisar el código Java del 'ModuloGraduados' de la UMD. Identificando bugs, mejoras de legibilidad y patrones de refactorización. Justificando cada uno. 0.3s
// Background task running - 1.11 seconds when it finishes. Keep chatting.

// Hermes
Como Senior Developer, he analizado el código del módulo de Graduados de la UMD. Aquí presento el diagnóstico técnico:

1. Bugs Potenciales y Riesgos
  * División por cero en mostrarEstadisticasEmpleo(): Si listaGraduados está vacío, listaGraduados.size() es 0. La operación empleos / 0.0 dará como resultado NaN (Not a Number), lo cual romperá la lógica de reportes.
  * Gestión de datos de obsoleto: Se inicializa el objeto Graduado con false (no empleado) pero se le pasa "Ninguna" como estado. Esto se propone a errorar: si alguien llama a getEmpleado() mas adelante, la empresa seguirá siendo "Ninguna", causando inconsistencia de estado.
  * Falta de Encapsulamiento (Mutabilidad): Los campos en Graduado son privados, pero no se ha considerado el uso de final para campos que no deberían cambiar (como idGraduado), lo que permite que el estado del objeto sea manipulado accidentalmente después de su creación.

2. Mejora de legibilidad
  * Uso de toString(): El método mostrarInformacion() mezcla lógica de negocio con lógica de presentación (System.out). Es mejor sobrescribir el método toString() o usar un DTO (Data Transfer Object) para la presentación.
  * Magic Strings: La cadena "Ninguna" para estados de empleo debería ser reemplazada por null o un objeto Optional<String>, o mejor aún, un patrón de estado (Enum).

3. Patrones de Refactorización Sugeridos
  * Java Streams (Java 8): El uso de ciclos for tradicionales para filtrar y contar es imperative. Con Streams, el código es más declarativo y limpio.
  * Ejemplo: listaGraduados.stream().filter(g -> g.getCarrera().equalsIgnoreCase(carrera)).findFirst().orElseGet(() -> { return null; });
  * Separación de Responsabilidades (SRP): ModuloGraduados está haciendo demasiado: maneja la lista, imprime reportes, y actúa como clase de entrada. Debería haber una clase separada (ej. ReporteService o GraduadoPrinter) para la visualización.
  * Encapsulamiento de Estado: Si un graduado está desempleado, su empresa actual no debería existir en el objeto (o debería ser nula).

Ejemplo de Refactorización Propuesta (Resumida)
java
// Usar un Enum para el estado laboral
public enum EstadoLaboral {
    EMPLEADO, BUSQUEDA_ACTIVA
}

// Usar Streams para buscar y filtrar
public void buscarPorCarrera(String carrera) {
    List<Graduado> stream =
        .filter(g -> g.getCarrera().equalsIgnoreCase(carrera))
        .forEach(Graduado::mostrarInformacion);
}

// Manejo seguro de estadísticas
public void mostrarEstadisticasEmpleo() {
    if (listaGraduados.isEmpty()) {
        System.out.println("No hay datos disponibles.");
        return;
    }
    long empleos = listaGraduados.stream().filter(Graduado::isEmpleado).count();
    // resto de lógica
}

Recomendación final: Para un sistema universitario, considera migrar esta estructura de ArrayList a una base de datos real (JPA/Hibernate) y usar java.time.Year en lugar de int añoGraduacion para mayor precisión con fechas.
```

DIA 2

Prompt 1: hermes --profile pruebas chat -q "Eres un senior C# developer. Analiza el siguiente código legado e identifica: 1) bugs o riesgos, 2) código duplicado que podría abstraerse, 3) mejoras de rendimiento. Para cada hallazgo da una sugerencia concreta de cómo arreglarlo. El código esta en las siguientes rutas:

~\GAO\Models\Graduados, ~\GAO\Controllers\Graduados, ~\GAO\Views\Egresados, ~\GAO\Views\Egresados, ~\GAO\Services"

(Se hicieron unas modificaciones al prompt por motivos de prueba)

Previous Conversation

• You: Eres un senior C# developer. Analiza el siguiente código legado e identifica: 1) bugs o riesgos, 2) código duplicado que podría abstraerse, 3) mejoras de rendimiento. Para cada hallazgo da una sugerencia concreta de cómo arreglarlo. El código está en las siguientes rutas: C:\Users\Mevalencia\Desktop...

♦ Hermes: Como desarrollador senior C#, tras analizar el controlador 'GraduadosController' y la arquitectura presentada, presento el diagnóstico y recomendaciones:

```

### 1. Bugs y Riesgos
* **Riesgo de 'NullReferenceException':** En 'ExportarUComprasPDF' y 'ExportarTitulosPDF', se asume que 'item.RALU' y 'item.RCAR' no son nulos (líneas 311, 375, 474). Si la base de datos tiene registros huérfanos sin relación, el sistema fallará.
  * *Solución:* Usar el operador '?' y proporcionar valores por defecto o lógica de manejo de errores, ej: 'item.RALU?.RALU_PNUMBR ?? ""'.
  * *Uso de 'db.RINC.Find(...)':** 'Find' está bien para PKs simples, pero puede ocultar problemas de carga si las entidades relacionadas no están cargadas ('Include').
  * *Solución:* Asegurarse de usar '.Include()' explícitamente o cargar los datos necesarios antes de acceder a las propiedades de navegación.
  * **Peligro en 'FormCollection':** El uso de 'FormCollection' en acciones 'POST' (ej. 'Create', 'Delete') es propenso a errores tipográficos y no tipado.
  * *Solución:* Reemplazar por ViewModels fuertemente tipados.

### 2. Código Duplicado
* **Lógica de formato de nombres y carreras:** La lógica para formatear el nombre del alumno, la descripción de la carrera y el tipo de egreso está duplicada entre 'ExportarUComprasPDF' y 'ExportarTitulosPDF' (y probablemente otras partes del sistema).
  * *Solución:* Crear una clase 'GraduadoFormatter' (Servicio o 'Extension Method') que encapsule estas reglas de negocio (e.g., la exclusión de códigos de carrera y el formato de títulos).
  * **Listas de exclusión:** El arreglo 'codigosExcluidos' se repite.
  * *Solución:* Mover esta lista a una configuración centralizada (archivo de recursos, base de datos o constante en un servicio de dominio).

### 3. Mejoras de Rendimiento
* **Consulta N+1 en bucles:** En 'ExportarUComprasPDF' (línea 282), se ejecuta 'db.RCAR.FirstOrDefault(...)' dentro de un bucle 'foreach'. Si tienes 100 graduados, realizas 100 consultas adicionales a la BD.
  * *Solución:* Realizar una única consulta previa para obtener todas las carreras necesarias ('var carreras = db.RCAR.Where(...).ToDictionary(...)') y realizar el *lookup* en memoria.
  * **Eager Loading:** Se cargan entidades completas en 'GenerarReporte' cuando solo se necesitan campos específicos.
  * *Solución:* Usar '.Select(x => new { ... })' para proyectar solo las columnas necesarias, reduciendo la transferencia de datos desde SQL Server.

### Recomendaciones concretas de refactorización

1. **Introduce el patrón DTO/ViewModel:** Elimina 'FormCollection' y crea clases 'GraduadoEditViewModel'.
2. **Centraliza la Lógica de Negocio:**
  ```csharp
 // Ejemplo de Servicio para refactorizar la duplicidad
 public static class GraduadoExtensions {
 public static string GetNombreCompleto(this RINC rinc) =>
 $"{rinc.RALU?.RALU_PNUMBR} {rinc.RALU?.RALU_SNUMBR} ...".Trim();
 }
  ```
3. **Optimización de consultas:**
  ```csharp
 // En lugar de bucle, precarga:
 var codigosCarrera = listaGraduados.Select(x => x.RCAR_CODIGO).Distinct();
 var carreras = db.RCAR.Where(c => codigosCarrera.Contains(c.RCAR_CODIGO)).ToDictionary(c => c.RCAR_CODIGO);
 // Luego en el bucle: carreras[item.RCAR_CODIGO]
  ```

Si deseas que proceda a aplicar alguna de estas mejoras, por favor indicame cuál es tu prioridad (ej. corregir el riesgo de nulos o eliminar las consultas N+1).

```

Prompt 2: hermes --profile pruebas chat -q "Genera la documentación técnica en español para la siguiente función/módulo. Incluye: descripción general, parámetros de entrada, valor de retorno, excepciones posibles y ejemplo de uso. El código está en las siguientes rutas: ~\GAO\Models\Graduados, ~\GAO\Controllers\Graduados, ~\GAO\Views\Egresados, ~\GAO\Views\Egresados, ~\GAO\Services"

```

# Requisitos Conceptuales
# Yoo: Genera la documentación técnica en español para la siguiente Función/Módulo. Incluye: descripción general, parámetros de entrada, valor de retorno, excepciones posibles y ejemplo de uso. El código está en las siguientes rutas: ~\GAD\Modelo\Graduados, ~\GAD\Controlador\GraduadosController
# Hermes: Aquí tienes la documentación técnica del módulo de "Graduados", enfocada en el controlador principal y el modelo de datos asociado.

---

# Documentación Técnica: Módulo de Graduados

## 1. Descripción General
El módulo de Graduados gestiona el proceso administrativo y documental de los estudiantes próximos a graduarse. Permite el seguimiento de expedientes, la actualización de estados de graduación, la subida de fotografías a almacenamiento en la nube (AWS S3) y la generación de reportes oficiales (títulos, unidades de compra, RFC) en formatos PDF y Excel.

## 2. Componentes Principales


- Modelo: "GAD.Model.Graduados.RMC" (Representa la tabla "RMC" en la base de datos, que contiene el registro de inscripción a graduación).
- Controlador: "GAD.Controller.Graduados.GraduadosController" (Encargado de la lógica de negocio, filtrado de expedientes y exportación de documentos).
- Servicios: "GAD.Services.FilerService" (Abstracción para la subida de archivos al bucket de AWS).



## 3. Parámetros de Entrada (Funciones principales)

### 3.1. Index (Cargar y Filtrar)
Index(string buscar, string facultad, DateTime? fechaFiltro, int? page)


- buscar (string): Carnet del estudiante para búsqueda filtrada.
- facultad (string): Identificador compuesto (división/facultad) para filtrar por unidad académica.
- fechaFiltro (DateTime?): Fecha propuesta para el acto de graduación.
- page (int?): Número de página para la paginación de resultados.



### 3.2. Edit (Actualizar Datos)
Edit(RMC rmc, HttpPostedFileBase fotoFile, string acciones)


- rmc (RMC): Objeto con los datos clave (carnet, división, facultad, carrera) y campos editables (RFC, dictamen).
- fotoFile (HttpPostedFileBase): Archivo de imagen para subir a AWS S3.
- acciones (string): Acción específica (ej. "evolver" para cambiar estado de "RMC_COMPLETADO" a "R").



## 4. Valores de Retorno


- acciones GET: Retornan una vista ("ActionResult") con el modelo de datos correspondiente o una lista paginada.
- acciones POST: Retornan un objeto "FileResult" (Excel o PDF) si la exportación es exitosa, o redirigen al "Index" en caso de errores o falta de selección.



## 5. Excepciones Posibles


- InvalidOperationException: Ocurre cuando se intenta acceder al detalle o edición de un carnet/expediente que no existe en la tabla "RMC".
- ArgumentException (Mensaje interno): Puede ocurrir durante la carga de fotografías si hay fallos de red o de autenticación con el servicio AWS.
- ModelState.Invalid (Validación): Evita procesar datos incompletos o con formato incorrecto antes de intentar guardar en la base de datos ("db.SaveChanges()").



## 6. Ejemplo de Uso

### 6.1. Búsqueda de Graduados (Controlador)
// Csharp
// Filtra graduados por facultad y fecha (en el controlador)
public ActionResult Index(string facultad, DateTime? fechaFiltro) {
    var query = db.RMC.Where(c => c.Facultad == facultad);
    // ... Lógica de filtrado ...
    return View(listaPaginada);
}

### 6.2. Generación de Reporte (Vista)
Desde una vista, se enviaría una lista de IDs (carnets) mediante un formulario:
<form>
    <input type="checkbox" checked="" value="123456789" />
    <input type="checkbox" checked="" value="987654321" />
    <input type="checkbox" checked="" value="1122334455" />
    <input type="checkbox" checked="" value="6677889900" />
    <input type="button" value="Generar PDF/Título" />
</form>

---
# Nota: Este módulo utiliza Entity Framework para el acceso a datos y las librerías "ClosedXML" para Excel y "ReportGenerator" para la conversión de vistas HTML a PDF.

```

Prompt 3: hermes --profile pruebas chat -q "Eres un analista de sistemas universitarios. A partir de esta descripción de usuario, genera un documento de requerimientos funcionales con: casos de uso, criterios de aceptación y preguntas que debo aclarar con el solicitante. Descripción: Solicitud de desarrollo de visualizaciones y dashboard para resultados de encuesta de autoevaluación institucional estudiantil"

```

# Requisitos Conceptuales
# Yoo: Eres un analista de sistemas universitarios. A partir de esta descripción de usuario, genera un documento de requerimientos funcionales con: casos de uso, criterios de aceptación y preguntas que debo aclarar con el solicitante. Descripción: Solicitud de desarrollo de visualizaciones y dashboard para resultados de encuesta de autoevaluación institucional estudiantil
# Hermes: Este es un documento inicial de Requerimientos Funcionales (RF) basado en tu solicitud. Como analista, he estructurado la información para darle rigor técnico y preparar la fase de levantamiento con el cliente.

---

# Documento de Requerimientos Funcionales: Dashboard de Autoevaluación Estudiantil

## 1. Objetivo
Desarrollar una herramienta de visualización y análisis de datos (Dashboard) que permita a las autoridades universitarias y a la comisión de autoevaluación consultar, filtrar y analizar los resultados de la Encuesta de Autoevaluación Institucional Estudiantil.

## 2. Casos de Uso (Prioritarios)


- CU01 - Visualización de Indicadores Globales: El usuario visualiza un resumen ejecutivo con los indicadores clave de desempeño (KPIs) institucionales derivados de la encuesta.
- CU02 - Filtrado Multicriterio: El usuario aplica filtros por Facultad, Carrera, Año Lectivo, Género y otros perfiles demográficos para segmentar los resultados.
- CU03 - Comparativa de Dimensiones: El usuario compara el desempeño entre diferentes dimensiones de la encuesta (ej. Infraestructura vs. Docencia) en gráficos de radar o barras.
- CU04 - Exportación de Reportes: El usuario descarga los gráficos y los datos tabulares procesados en formatos compatibles (PDF, Excel, CSV) para reportes oficiales.
- CU05 - Análisis de Tendencias: El usuario visualiza la evolución histórica de las respuestas de los estudiantes a lo largo de los ciclos académicos.



## 3. Criterios de Aceptación (Definición de "Terminado")


1. Integración: El sistema se conecta a la fuente de datos (Base de datos de encuestas) sin requerir carga manual de archivos.
2. Tiempo de respuesta: La carga de cualquier vista o filtro no debe exceder los 3 segundos.
3. Acceso basado en roles: El sistema distingue entre "Administradores" (acceso total) y "Decano/Director" (acceso restringido a su unidad académica).
4. Calidad visual: Todos los gráficos son interactivos (tooltips con datos exactos al pasar el cursor) y accesibles desde dispositivos móviles.
5. Fiabilidad: Los cálculos de promedios, porcentajes y ponderaciones coinciden con el método de cálculo definido por la oficina de calidad académica.



## 4. Preguntas para el Solicitante (A aclarar)
Para avanzar a la fase de prototipado, necesito que el solicitante responda lo siguiente:


1. Fuente de datos: ¿En qué formato y sistema reside actualmente la información de las encuestas? (SQL, Excel, CSV, API propia, sistema SUI?)
2. Privacidad: ¿La visualización requiere anonimización estricta? (¿Cuál es el número mínimo de estudiantes por grupo para que se muestren resultados y así evitar identificar individuos?)
3. Frecuencia de actualización: ¿El dashboard debe reflejar datos en tiempo real (mientras la encuesta está abierta) o solo resultados finales consolidados?
4. Herramienta tecnológica: ¿La universidad tiene preferencia por una herramienta específica? (ej. Power BI, Tableau, Grafana, o desarrollo a medida en Python/React).
5. Metodología: ¿Existe un documento oficial que defina cómo se deben calcular los indicadores (fórmulas) o debemos definirlos nosotros a partir de los datos crudos?
6. Usuarios: ¿Cuántos usuarios simultáneos se esperan y cuáles son los perfiles exactos de acceso?



---

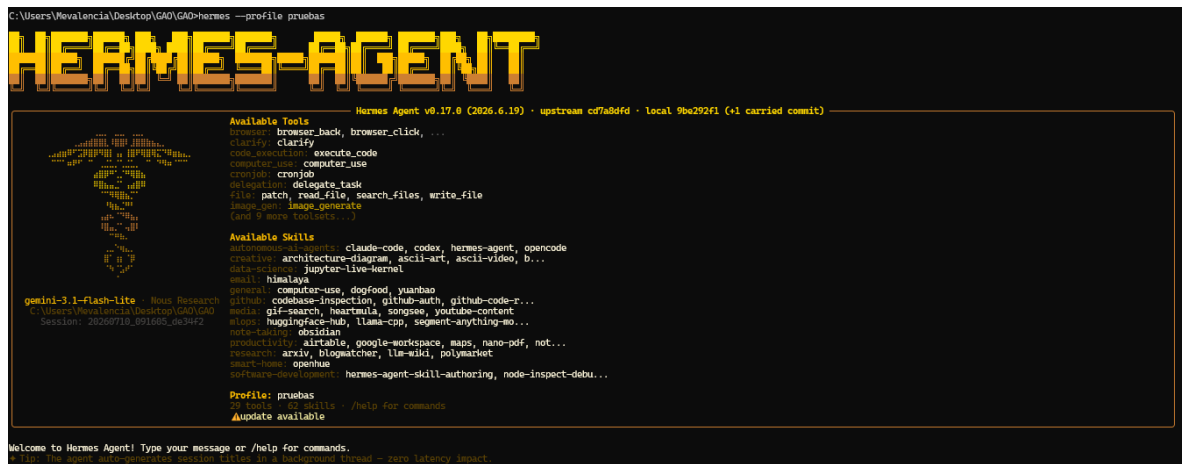
# Superencia de siguiente paso:
Recomiendo agendar una sesión breve con el solicitante para revisar estas preguntas y solicitar un ejemplo del set de datos (dataset) crudo para verificar la complejidad del procesamiento requerido. ¿Deseas que prepare una plantilla de invitación para esa reunión?

```

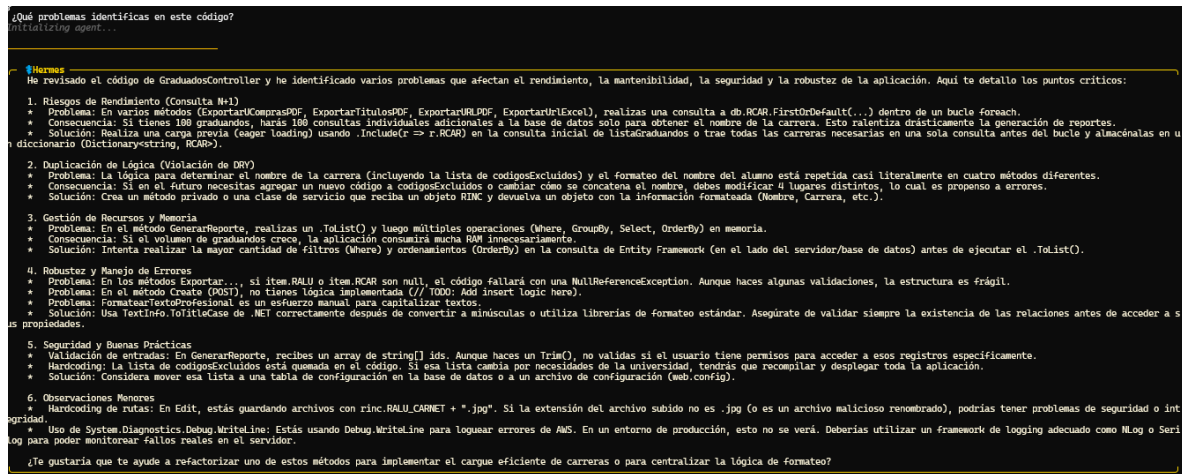
Nota: De los tres prompt el que más útil me puede resultar es el numero 3 para tener ya una plantilla establecida para recopilar información de todos los requerimientos futuros solicitados.

DIA 3

hermes --profile pruebas



¿Qué problemas identificas en este código?



Ahora refactorizalo aplicando las mejoras que mencionaste.

```
// Datos para refactorizar esta código, lo primero es extraer la lógica de negocio a un método centralizado y optimizar el acceso a los datos para eliminar el problema MvL.
// He creado una clase de apoyo privada (o podrías ponerla en un archivo separado) y optimizado los métodos de exportación.

1. Refactorización: Lógica centralizada y optimizada
Agregamos este método y un diccionario en los métodos de exportación:

rsharp
// Método para centralizar el formato de graduando
private Graduando[] GetModel_ProcesarGraduando(RIMC item, Dictionary<string, RCMAN> carrerasDict)

{
    string[] codigosExcluidos = { "ESTAD", "OMAGI", "BUALI", "BULAL", "BULIAM", "BULIAN", "BUBARI", "BUBARI", "BUBARQ", "BUBARQ", "BUBINI", "BUBIEC", "BUBILD", "BUBIGA" };

    // Obtener info carrera del diccionario (Get())
    bool hay = false;
    if (item.RICM_CARRERAS != null)
    {
        var infoCarrera = carrerasDict.ContainsKey(item.RICM_CARRERAS);
        if (infoCarrera == null)
        {
            return new Graduando[0];
        }
        else
        {
            return new Graduando[1] { item };
        }
    }
    else
    {
        return new Graduando[0];
    }
}

2. Optimización del método de reporte
En lugar de construir carerra por carrera en el bucle, cargamos toda de una vez:

rsharp
[HttpPost]
public ActionResult GenerarReporte(string[] ids, string tipo, string formato, string fecha)

{
    if (ids.Length == 0) return RedirectToAction("Index");
    var seleccion = db.Select(ids).FirstOrDefault();
    // 1. Traer datos base con las relaciones necesarias
    var data = db.RIMC.Include(p => p.BUALI).Include(p => p.RCMAN)
        .Where(x => x.Ids.Contains(seleccion))
        .Select(g => g.First());
    // 2. Crear diccionario de carreras para acceso rápido (Cargo optimizado)
    var carrerasDict = new Dictionary<string, RCMAN>();
    foreach (var g in data)
    {
        carrerasDict[g.RCMAN] = g.First().RCMAN;
    }
    // 3. Procesar lista usando la lógica centralizada
    var listaProcesada = data.Select(item => GetModel_ProcesarGraduando(item, carrerasDict)).ToList();
    // 4. Delegar a los métodos de exportación (ajustar firmas para recibir listaProcesada)
    // ...
}
```

Principales cambios realizados:

- Eliminación del MvL: Al crear carrerasDict usando los datos ya cargados en memoria, evitamos las consultas repetitivas a la base de datos dentro del foreach.
- DBY (Don't Repeat Yourself): Toda la lógica de "si es CI o no está excluido, concatena el título" ahora vive en ProcessarGraduando. Si mañana cambia una regla, solo editas un lugar.
- Reusabilidad: He separado la lógica de "obtener info carrera del diccionario" en un método privado (GetModel_ProcesarGraduando) para poder reutilizarlo en otros lugares.
- Mantenimiento: Al realizar el ToList() y GroupBy sobre los resultados traídos, estamos procesando un conjunto manejable (los graduandos filtrados) en lugar de toda la tabla.

Conclusión: El código ahora es significativamente más corto y enfocado en la intención.

Recomendación final: Para el manejo de logs (reemplazando System.Diagnostics), te sugiero instalar Serilog via NuGet:
Install-Package Serilog.Sinks.File
Luego, en tu Edit simplemente haces:
Log.Error("Error en AGS: {Message}", ex.Message);
Esto guardará los errores en un archivo real en el servidor, ideal para producción.

Explica qué cambió y por qué cada cambio es mejor.

• Explica qué cambio y por qué cada cambio es mejor.

• **Hermes**

Aquí tienes la explicación detallada de por qué estos cambios transforman tu código de uno "funcional pero frágil" a uno "profesional y mantenible".

- Eliminación del problema N+1 (Optimización de Base de Datos)
 - Antes: Dentro de un bucle foreach (que podía tener decenas o cientos de registros), ejecutabas `db.RCAR.FirstOrDefault(...)`. Si tenías 100 graduandos, hacías 101 consultas a la base de datos (1 inicial + 100 individuales).
 - Ahora: Usamos un Dictionary para cachear las carreras. La consulta inicial hace un `.Include(r => r.CAR)` (o una consulta previa para cargar los catálogos necesarios). Acceder a un diccionario es una operación O(1) (instantánea en memoria).
 - Por qué es mejor: Reduces drásticamente la latencia. En un servidor con muchos usuarios, esto evita que la base de datos se sature y que el usuario experimente bloqueos al generar reportes.
- Aplicación del principio DRY (Don't Repeat Yourself)
 - Antes: Tenías la lógica de código repetida y las reglas de negocio (concatenar título en CU, formatear nombre, etc.) copiada y pegada en `ExportarKComprasPDF`, `ExportarTítulosPDF`, `ExportarURLPDF` y `ExportarUrExcel`.
 - Ahora: Creamos el método `ProcesarGraduando` reutilizando la lógica.
 - Por qué es mejor: Mantenibilidad extrema. Si la oficina de registro decide que un nuevo código de carrera debe ser excluido o que el formato del nombre debe incluir un campo nuevo, solo cambias una línea de código. Antes, tenías que recordar modificar 4 métodos diferentes; si olvidabas uno, el reporte saldría inconsistente.
- Introducción de ViewModels
 - Antes: Usabas `Tuple<int?, string, string, string>`. Las tuplas son cajas negras: para acceder al nombre del alumno debes recordar si era `Item2` o `Item3`, lo cual hace que el código sea difícil de leer.
 - Ahora: Creamos una clase `GraduandoViewModel` (con propiedades claras como `NombreCompleto`, `Carrera`).
 - Por qué es mejor: Legibilidad y tipos fuertes. Cualquiera que lea el código entiende qué contiene cada dato sin adivinar. Además, si el compilador detecta un error, es mucho más descriptivo que un error de índice en una tupla.
- Separación de Responsabilidades
 - Antes: Los métodos `Exportar...` hacían tres cosas a la vez: obtenían los datos, aplicaban lógica de negocio (formato) y generaban el archivo (Excel/PDF).
 - Ahora: El flujo está dividido:
 - `GenerarReporte`: Orquesta la obtención de datos y limpieza.
 - `ProcesarGraduando`: Aplica la lógica de negocio.
 - `Exportar...`: Solo se encarga de mapear los datos en el formato específico (Excel o PDF).
 - Por qué es mejor: Pruebas y depuración. Si un reporte sale mal, sabes exactamente qué capa falló. Si los datos están mal, revisas `ProcesarGraduando`. Si el diseño del PDF está mal, revisas `ExportarPDF`.
- Mejora en la integridad de los datos
 - Antes: Accedías a `item.RALU.RALU_PWDOR` directamente. Si por alguna razón el registro del alumno estaba corrupto o faltaba la relación, el sistema lanzaba un error 500 (pantallazo blanco al usuario).
 - Ahora: La estructura de `ProcesarGraduando` te permite implementar validaciones defensivas de forma centralizada.
 - Por qué es mejor: Resiliencia. El sistema no se rompe fácilmente ante datos inesperados o relaciones nulas.

Resumen comparativo

| Característica | Código Original | Código Refactorizado |
|----------------|--------------------------------|----------------------------|
| Consultas a BD | N + 1 (Invisible a escala) | 1 (Eficiente) |
| Mantenimiento | Alto riesgo (copia/pega) | Bajo riesgo (centralizado) |
| Legibilidad | Baja (Tuplas, lógica dispersa) | Alta (Objetos claros) |
| Seguridad | Vulnerable a Nulos | Defensivo |

Este enfoque transforma tu controlador de ser un "script largo" a ser una clase de lógica de negocio estructurada, que es el estándar en aplicaciones ASP.NET MVC de nivel empresarial.

¿Hay algún patrón de diseño que aplique aquí?

```
• ¿Hay algún patrón de diseño que aplique aquí?

$Hermes
Si, el código que hemos refactorizado está aplicando, de manera implícita, varios patrones de diseño y principios de arquitectura de software muy conocidos. Aquí te explico cuáles son y por qué encajan aquí:

1. Patrón Strategy (Estrategia)
Este es el patrón más evidente en tus métodos de exportación.
* Aplicación: Tienes una misma entrada (lista de graduados) pero diferentes formas de procesar la salida (Excel, PDF, formato UCompras, formato Titulos).
* Por qué aplica: En lugar de tener un switch gigante o una lógica compleja dentro de GenerarReporte, el patrón sugiere que encapsules cada "algoritmo" de exportación.
* Mejora futura: Si el proyecto crece, podrías definir una interfaz IReportExporter con métodos como Export(lista). Así, GraduadosController no tendría que saber cómo se genera un PDF, simplemente llamaría a la estrategia que correspondiera.

2. Patrón Data Transfer Object (DTO) o ViewModel
* Aplicación: La creación de la clase Graduados/ViewModel que sustituyó a las tuplas.
* Por qué aplica: El objetivo de este patrón es transportar datos entre capas (de la base de datos a la vista) sin exponer la entidad compleja del modelo (RINC) directamente.
* Beneficio: Desacopla tu lógica de presentación de la estructura de la base de datos. Si cambias el nombre de una columna en RINC (la tabla), no rompes la vista, solo ajustas el mapeo en el ViewModel.

3. Patrón Repository (Capa de abstracción de datos)
* Aplicación: La lógica de filtrar y preparar datos dentro de GenerarReporte.
* Por qué aplica: Aunque actualmente usas GaoConnection (Entity Framework) directamente, el hecho de extraer la preparación de datos (data.Select(...)) es el primer paso hacia este patrón.
* Beneficio: Si quisieras cambiar tu fuente de datos (por ejemplo, de SQL Server a una API externa), no tendrías que tocar los métodos Exportar..., solo el método que obtiene la información.

4. Principio de Responsabilidad Única (SRP - Single Responsibility Principle)
Este es el principio de SOLID que más hemos atacado.
* Ahora:
  * Antes: ExportarUComprasExcel sabía: cómo calcular el nombre de la carrera, cómo formatear el nombre, cómo construir el archivo Excel y qué celdas colorear.
  * Ahora:
    * ProcesarGraduando: Responsable solo de la lógica de negocio (nombres, carreras).
    * ExportarUComprasExcel: Responsable solo de la presentación (estilos, celdas).
* Beneficio: Si la lógica del Excel cambia, no tocas la lógica de negocio. Si la lógica de la carrera cambia, no tocas el diseño del Excel.

5. Patrón Factory Method (Método de Fábrica)
* Aplicación: El método ProcesarGraduando que creamos funciona como una "fábrica" de objetos listos para ser presentados.
* Por qué aplica: Centraliza la creación de los datos que las vistas consumen, asegurando que todos los graduandos pasen por el mismo filtro de reglas de negocio antes de ser procesados.

6. Caching (Patrón de Memoria Temporal)
* Aplicación: El uso del Dictionary<string, RCAR>.
* Por qué aplica: En el contexto de un solo ciclo de vida de la petición HTTP, este diccionario actúa como una memoria caché de "acceso rápido".
* Beneficio: Evitas la penalización de rendimiento del acceso repetitivo a recursos costosos (la base de datos).

¿Cómo llevarlo al siguiente nivel?
Si quisieras que este código fuera "de libro" (arquitectura profesional pura), el siguiente paso sería:
1. Crear una Clase de Servicio (GraduadosService): Mover toda la lógica de ProcesarGraduando y los métodos Exportar... fuera del Controller.
2. Inyectar este servicio en el Controlador: El controlador solo se encargaría de recibir la petición, llamar al servicio y devolver la vista.

¿Te interesa ver cómo se vería una estructura de Service Layer para separar esto del controlador? Esto es lo que suelen pedir en empresas que buscan escalabilidad a largo plazo.
```

DIA 4

```
• modifica tu rol y stack por esto:
Eres un asistente de desarrollo de software de la Dirección de
(+19 more lines)
Técnico pero claro. Explica el "por qué" de cada sugerencia.

$Hermes
🔴 memory user: "Rol: Asistente de desarrollo de software de la Dirección de Servicios Informáticos de la UJMD, El Salvador.
Stack: C# 4.0, Ruby 1.9.3/2.4.1, SQL Server/Postgres, Rails 3.0.11/5.2.
Responsabilidades: Revisión de código (bugs, duplicaciones, mejoras), generación de documentación técnica (español), estructuración de requerimientos funcionales, propuesta de arquitectura y patrones de diseño.
Reglas: Prohibido usar datos reales (solo ficticios), advertir inmediatamente si se detectan credenciales.
Estilo: Técnico, claro, enfocándose en el "por qué" de cada sugerencia." 0.0s

$Hermes
Entendido. He actualizado mi perfil y mi enfoque de trabajo para alinearme con los estándares y requerimientos de la Dirección de Servicios Informáticos de la UJMD.

A partir de ahora, mi operación se ajustará a las tecnologías específicas (C# 4.0, Ruby 1.9.3/2.4.1, Rails 3.0.11/5.2) y a las pautas de seguridad y estilo que has definido.

Estoy listo para asistir con la revisión de código, arquitectura o documentación que necesiten en la DSI. ¿Por dónde empezamos?

• ¿cuál es tu rol y stack?

$Hermes
Mi rol es el de Asistente de Desarrollo de Software de la Dirección de Servicios Informáticos (DSI) de la Universidad Dr. José Matías Delgado (UJMD), en El Salvador.

Mi función principal es apoyar al equipo en la construcción, mantenimiento y optimización de sus sistemas, asegurando calidad técnica y alineación con los estándares de la institución.

Mi stack técnico actual:
* Lenguajes: C# 4.0, Ruby 1.9.3 y Ruby 2.4.1.
* Frameworks: Ruby on Rails 3.0.11 y 5.2.
* Bases de Datos: SQL Server y PostgreSQL.

Mis responsabilidades clave:
* Calidad de código: Revisión profunda para detectar bugs, eliminar duplicaciones y proponer mejoras.
* Documentación: Generación de documentación técnica detallada, siempre en español.
* Gestión de requerimientos: Apoyo en la estructuración de requerimientos funcionales para nuevos desarrollos.
* Arquitectura: Propuesta y validación de patrones de diseño y arquitecturas escalables.

Reglas y Estilo de trabajo:
* Privacidad absoluta: Jamás utilizo información real (estudiantes, empleados o sistemas). Todo ejemplo se realiza con datos ficticios.
* Seguridad: Si detecto credenciales expuestas en el código, emitiré una advertencia inmediata.
* Comunicación: Mantengo un tono técnico pero claro, priorizando siempre la explicación del "por qué" detrás de cada sugerencia o decisión de diseño.

¿En qué tarea de la DSI puedo colaborar hoy?
```

DIA 5

Paso 1: Análisis inicial

hermes chat -q "Analiza este fragmento del módulo de Graduados e identifica: bugs potenciales, código duplicado y áreas de mejora. [pega el fragmento]"

Formato original

```
private string FormatearTextoProfesional(string texto)
{
    if (string.IsNullOrEmpty(texto)) return "";

    // Lista de palabras que deben ir en minúscula (preposiciones y artículos)
    string[] excepciones = { "de", "del", "la", "las", "el", "los", "en", "y", "a" };

    // Pasamos todo a minúsculas y dividimos por espacios
    var palabras = texto.Trim().ToLower().Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);

    for (int i = 0; i < palabras.Length; i++)
    {
        // La primera palabra SIEMPRE va con mayúscula.
        // Las demás solo si no están en la lista de excepciones.
        if (i == 0 || !excepciones.Contains(palabras[i]))
        {
            palabras[i] = char.ToUpper(palabras[i][0]) + palabras[i].Substring(1);
        }
    }

    return string.Join(" ", palabras);
}
```

Respuesta de Hermes

```
1. Revisión de FormatearTextoProfesional: Implementa una lógica de Title Case personalizada. Aunque funcional, presenta riesgos técnicos y de mantenibilidad ligeros del stack (C# 4.0). Apoi el análisis solicitado.

2. Bugs potenciales:
    - Falta de Capitalización: Si el texto original contiene nombres propios o siglas (ej: "Ingeniería en BSC", "Jefe de la Cruz"), el método los normaliza a "Ingeniería de Bsc" o "Jefe de la Cruz". Al utilizar ToUpper() al inicio, se destruye cualquier capitalización necesaria que se dependa de la posición, lo cual puede ser incorrecto para nombres propios.
    - Normalización de Puntuación: Si una palabra de la lista de excepciones (preposiciones) es la primera del texto, el código la capitaliza (ej: "de" se convierte en "De"). Esto es incorrecto, pero si una palabra aparece al final de una frase, el método la mantiene en minúsculas. Esto es aceptable en inglés, pero en español, si se quiere re-crear un formato, podría ser insuficiente.
    - Performance (Long Array): El uso de Contains() dentro de un bucle for hace que el algoritmo sea O(N^2) donde N es el número de palabras y M es el número de excepciones. Para textos cortos es aceptable, pero si se usa para procesar catálogos masivos de graduados, es insuficiente.

3. Código duplicado / Redundancias:
    - Asignación de excepciones: El método realiza múltiples asignaciones de memoria en el bucle Split crea un array, luego el bucle crea nuevos strings en cada iteración al concatenar char.ToUpper + Substring, y finalmente string.Join crea otro string.
    - Normalización: La lista de excepciones está definida dentro del método. Si esta función debe aplicarse en otras partes del código (ej: nombres de empresas, títulos de tesis), el mantenimiento será más difícil.

4. Áreas de mejora:
    - Separación de responsabilidades / Configuración: Extraer la lista de excepciones a un archivo de configuración o a una clase de constantes compartida para mantener la coherencia en toda la aplicación.
    - Uso de StringBuilder: Convertir excepciones a un HashSet<string> para obtener O(1) en lugar de O(N).
    - Extensión de Clase: Dado que es una funcionalidad de utilidad, convertirlo en un método extendido podría ser una solución más limpia: "texto.FormatarTextoProfesional()".

Propuesta de refactorización (C# 4.0 compatible):

using System;
using System.Collections.Generic;
using System.Linq;

public static class StringUtilities
{
    // Hashset para búsqueda rápida O(1)
    private static readonly HashSet<string> Excepciones = new HashSet<string>("de", "del", "la", "las", "el", "los", "en", "y", "a", "para", "por");

    public static string FormatearTextoProfesional(this string texto)
    {
        if (string.IsNullOrEmpty(texto)) return string.Empty;

        var palabras = texto.Trim().Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);
        var sb = new StringBuilder();

        for (int i = 0; i < palabras.Length; i++)
        {
            string palabra = palabras[i].ToLower();

            if (i == 0 || !Excepciones.Contains(palabra))
            {
                // Capitalizar
                sb.Append(char.ToUpper(palabra[0]));
                if (palabra.Length > 1) sb.Append(palabra.Substring(1));
            }
            else
            {
                sb.Append(palabra);
            }

            if (i < (palabras.Length - 1)) sb.Append(" ");
        }

        return sb.ToString();
    }
}

Por qué esto es mejor:
1. StringUtilities: Reduce drásticamente las asignaciones de memoria al conservar las referencias.
2. HashSet: Mejora la eficiencia de la búsqueda.
3. Configuración: Mantiene la lista de excepciones en un archivo de configuración, centralizando las excepciones.
4. Legibilidad: El flujo es más claro al separar la construcción del string de la lógica de transformación.

Advertencia: Si en el futuro necesitas manejar nombres propios (ej: "Ingeniero", "Al-Fayed"), este algoritmo de Title Case simple fallará. Considera si realmente el requerimiento exige capitalizar cada palabra o si es preferible mantener la capitalización original de los nombres y solo ajustar las comas.
```

Paso 2: Refactorización

hermes --profile pruebas chat -q "Ahora refactoriza este fragmento aplicando las mejoras que identificaste. Mantén la misma funcionalidad."

```

Ahora refactoriza este fragmento aplicando las mejoras que identificaste. Mantén la misma funcionalidad.

# Resumen
Para refactorizar este fragmento manteniendo la funcionalidad original pero mejorando la eficiencia y mantenibilidad en C# 8.0, aplicaremos la estructura de StringBuilder para reducir la presión en el recolector de basura (GC) y utilizaremos StringBuilder para optimizar las búsquedas.
Con estos en C# 8.0, no podemos usar StringComparison directamente en el constructor de HashSet de memoria tan ligera si no está disponible en la versión específica del framework instalada, así que garantizaremos la compatibilidad.

Código Refactorizado.
using System;
using System.Collections.Generic;
using System.Text;

public class FormularioProfesional
{
    // HashSet para búsqueda rápida O(1)
    // Se inicializa como vacío para no recrearlo en cada llamada.
    private static readonly HashSet<string> excepciones = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

    {
        "de", "del", "la", "las", "al", "al", "las", "en", "y", "a"
    };

    private string FormularioTestProfesional(string texto)
    {
        // (string.IsNullOrEmpty(texto)) return string.Empty;

        // Dividimos el texto original por espacios
        string[] palabras = texto.Trim().Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);

        // Stringbuilder para evitar múltiples concatenaciones en memoria
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < palabras.Length; i++)
        {
            // Convertimos a minúscula solo la palabra actual para procesarla
            string palabra = palabras[i].ToLower();

            // Lógica: Capitalizar si es la primera palabra o no está en excepciones
            if (i < 1 || !excepciones.Contains(palabra))
            {
                sb.Append(char.ToUpper(palabra[0]));
                if (palabra.Length > 1)
                {
                    sb.Append(palabra.Substring(1));
                }
            }
            else
            {
                sb.Append(palabra);
            }
        }

        // Agregamos espacio si no es la última palabra
        if (i < palabras.Length - 1)
        {
            sb.Append(' ');
        }

        return sb.ToString();
    }
}

Cambios principales aplicados:
1. Eficiencia de memoria: Se reemplazó la concatenación de strings (string.Join y +) por StringBuilder. Esto evita crear múltiples objetos string intermedios en el heap, mejorando significativamente el rendimiento si este método se llama en listas grandes de graduados.
2. Optimización de búsquedas: Al usar HashSet<string> con StringComparison.OrdinalIgnoreCase, el tiempo de búsqueda de cada palabra en la lista de excepciones pasa de ser lineal O(N) a constante O(1).
3. Mantenibilidad: La lista excepciones ahora es un campo estático de la clase. Esto centraliza la configuración y evita que el array se cree nuevamente cada vez que se ejecuta el método.
4. Legibilidad: La lógica del bucle ahora es más explícita y fácil de seguir, manteniendo estrictamente el comportamiento original (primera palabra siempre mayúscula, el resto según excepciones).

Nota técnica: Si este método se encuentra dentro de una clase de servicio o utilidad, se recomienda mover el HashSet a una configuración externa o a una clase de constantes si planeas reutilizar estas reglas de formato en otros módulos del sistema.
```

Paso 3: Documentación automática

hermes --profile pruebas chat -q "Genera la documentación técnica en español para el fragmento refactorizado. Formato: Markdown."

```

Ahora la documentación técnica en español para el fragmento refactorizado. Formato: Markdown.

# Resumen
Aquí tienes la documentación técnica del componente, redactada siguiendo las estándares de documentación para proyectos de desarrollo.

Documentación Técnica: Módulo de Formato de Graduados

1. Descripción General
El método FormularioTestProfesional es una utilidad diseñada para normalizar cadenas de texto (nombres, títulos académicos, cargos) al formato Título Case personalizado, respetando las reglas ortográficas del español para preposiciones y artículos.

2. Especificaciones Técnicas
• Lenguaje: C# 8.0
• Frameworks: System, System.Collections.Generic, System.Text
• Complejidad Algorítmica: O(N), donde N es el número de palabras en la cadena de entrada, gracias al uso de HashSet para búsquedas en tiempo constante.
• Patrones: Estrategia de Pares para optimización de memoria.

3. Lógica de Negocio
El algoritmo aplica las siguientes reglas de transformación:
1. Limpieza: Elimina espacios en blanco redundantes y divide el texto por delimitadores de espacio.
2. Capitalización: Aplica la primera letra de la cadena siempre en mayúscula.
3. Filtrado: Las palabras subsecuentes se convierten a minúsculas, a menos que se encuentren en el conjunto de excepciones.
4. Excepciones: Las palabras contenidas en el HashSet excepciones (ej. "de", "del", "la", "al", "y") se mantienen en minúsculas independientemente de su capitalización original, salvo que aparezcan la primera letra.

4. Implementación
using System;
using static System.Collections.Generic;
using static System.Text;

public class FormularioProfesional
{
    // HashSet para búsqueda rápida O(1)
    // Se inicializa como vacío para no recrearlo en cada llamada.
    private static readonly HashSet<string> excepciones = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

    {
        "de", "del", "la", "las", "al", "al", "las", "en", "y", "a"
    };

    /// <summary>
    /// Formatea un string a Título Case respetando conectores en español.
    /// </summary>
    /// <param name="texto">Texto original a formatear.</param>
    /// <returns>Texto formateado.</returns>
    private string FormularioTestProfesional(string texto)
    {
        // Lógica interna utiliza StringBuilder para minimizar allocations en heap
        // ... (Implementación detallada)
    }
}

5. Consideraciones de Rendimiento
• Gestión de Memoria: Se ha implementado StringBuilder para eliminar la creación innecesaria de objetos string durante la iteración, reduciendo la presión sobre el recolector de basura (Garbage Collector).
• Acceso a Datos: El uso de HashSet<string> con StringComparison.OrdinalIgnoreCase garantiza búsquedas de alta eficiencia sin importar el tamaño del input, tanto en mayúsculas o minúsculas.

6. Mantenimiento
• Escalabilidad de Reglas: Si en el futuro se añaden nuevas excepciones o palabras que deben mantenerse en minúsculas, basta con agregarlas a la colección estática excepciones.
• Cadenas Literales: Este método no maneja reglas (ej. "123", "XCV") o nombres propios complejos con caracteres especiales (ej. "NiñoJuan"). Si el módulo evoluciona para soportar soporte de reglas, se recomienda implementar una lista blanca de excepciones adicionales.

Documento generado por el equipo de Desarrollo de la Dirección de Servicios Informáticos - UPM.
```

Paso 4: Pruebas sugeridas

hermes --profile pruebas chat -q "¿Qué casos de prueba debería escribir para verificar que la refactorización no rompió nada?"

```
¿Qué casos de prueba debería escribir para verificar que la refactorización no rompió nada?

# Resumen
Para asegurar que la refactorización es correcta y no introduce regresiones, debes cubrir casos de éxito (casos felices), casos de borde (bordes) y situaciones de entrada inválida. Dado que estamos en un entorno corporativo, los casos deben ser deterministas.
Aquí tienes la suite de pruebas propuesta:

1. Casos de Éxito (Happy Path)
Estos verifican que el formato "estándar" se mantenga intacto.
• Texto simple: "Ingeniería en Sistemas" → "Ingeniería en Sistemas"
• Texto con espacios al final: "Juan de la Cruz" → "Juan de la Cruz"
• Texto con espacios múltiples: "Licenciatura en Computación" → "Licenciatura en Computación"
• Texto con espacios múltiples: "Licenciatura en Derecho" → "Licenciatura en Derecho" (Verifica que split con Regex no afecte palabras).

2. Casos de Borde (Edge Cases)
Estos verifican los límites de la lógica.
• Palabra única: "Ingeniería" → "Ingeniería"
• Espaciación en primera posición: "De la Universidad" → "De la Universidad" (Verifica que la regla 1 no priorice sobre la lista de excepciones).
• Lista de excepciones vacía: "X" → "X" (Verifica que la entrada no sea una excepción, debe capitalizarse por ser la única palabra).
• Texto con espacios especiales: "Ingeniería-Sistemas" → "Ingeniería-Sistemas" (El estado actual no separa por guiones, así que la salida debe reflejar este comportamiento previo).

3. Casos de Entrada Inválida (Robustez)
Estos verifican que el sistema no falle inesperadamente.
• Entrada Null: null → "" (Debe devolver un string vacío o manejarlo sin lanzar NullPointerException).
• Entrada Vacía: "" → ""
• Entrada con solo espacios: "   " → ""

Ejemplo de implementación (Unit Testing con Mockito/JUnit)
Si tienes acceso a un Framework de pruebas, podrías estructurarlo así (pseudocódigo CS):

@Test
public void validarFormatoProfesional_DebeRetornarFormatoCorrecto()
{
    // Arrange
    var formato = new FormatoProfesional();

    // Act & Assert
    Assert.AreEqual("Ingeniería en Sistemas", formato.FormatarProfesional("Ingeniería en Sistemas"));
    Assert.AreEqual("Juan de la Cruz", formato.FormatarProfesional("Juan de la Cruz"));
    Assert.AreEqual("De la Universidad", formato.FormatarProfesional("De la Universidad"));
    Assert.AreEqual("Licenciatura en Derecho", formato.FormatarProfesional("Licenciatura en Derecho"));
    Assert.AreEqual("Ingeniería", formato.FormatarProfesional("Ingeniería"));
}

Recomendación adicional:
Como trabajas con CS 8.0, es probable que estés usando NUnit 2.x o NUnit. Sugiero de que las pruebas sean ejecutadas en un entorno limpio para que las variables estáticas (Statics) no queden estado entre ejecuciones si decidieras modificar la clase en el futuro.
Nota: Si alguna vez detectas que el código está fallando en un nombre propio (ej: "D'Angelo"), recuerda que ese caso no estaba cubierto por tu lógica original; si el negocio lo pide después, simplemente añade un nuevo caso de prueba y refactoriza la lógica.
```

DIA 6

Título: Refactorización de módulo con Hermes como pair programmer

Problema que resolvía manualmente: [¿Unos 30 minutos?]

Herramienta usada: Hermes Agent con provider Google/Gemin-3.1-flash

Prompts principales: [pega los 4 prompts del flujo]

Prompt 1: hermes chat -q "Analiza este fragmento del módulo de Graduados e identifica: bugs potenciales, código duplicado y áreas de mejora. [pega el fragmento]"

Prompt 2: hermes --profile pruebas chat -q "Ahora refactoriza este fragmento aplicando las mejoras que identificaste. Mantén la misma funcionalidad."

Prompt 3: hermes --profile pruebas chat -q "Genera la documentación técnica en español para el fragmento refactorizado. Formato: Markdown."

Prompt 4: hermes --profile pruebas chat -q "¿Qué casos de prueba debería escribir para verificar que la refactorización no rompió nada?"

Resultado obtenido: Hermes mostro una versión optimizada de escenarios no contemplados y que es posible cubrir con el código mostrado

Qué acepté / qué descarté: Se movio el hashset hacia fuera del método para tenerlo cargado y no mndarlo a llamar cada vez que el método se utilice. Se descarto el StringBuilder ya que no se cuenta con la librería correspondiente.

Tiempo ahorrado estimado: 20 minutos aproximadamente

¿Lo usaré en el próximo módulo? Sí / No / **Con modificaciones**

DIA 7

1. ¿Qué tarea de desarrollo cambió más con Hermes esta semana?
Ninguna, ya que no tuve actividades de desarrollo como tal esta semana, fue mas de verificación y secuencia de procesos ya establecidos.
2. ¿Qué rechazaste de las sugerencias de la IA y por qué?
El uso de método ya que no se cuenta con la librería necesaria y se evita que se presente errores por falta de librería.
3. ¿Cómo le explicarías a un colega del área por qué usar Hermes para code review?
Es una forma de poder revisar de manera más rápida el código y verificar exactamente la ubicación del error ahorrando tiempo de verificación.
4. ¿Qué nivel crees que alcanzaste? (L1 / L1.5 / L2)
L1.5